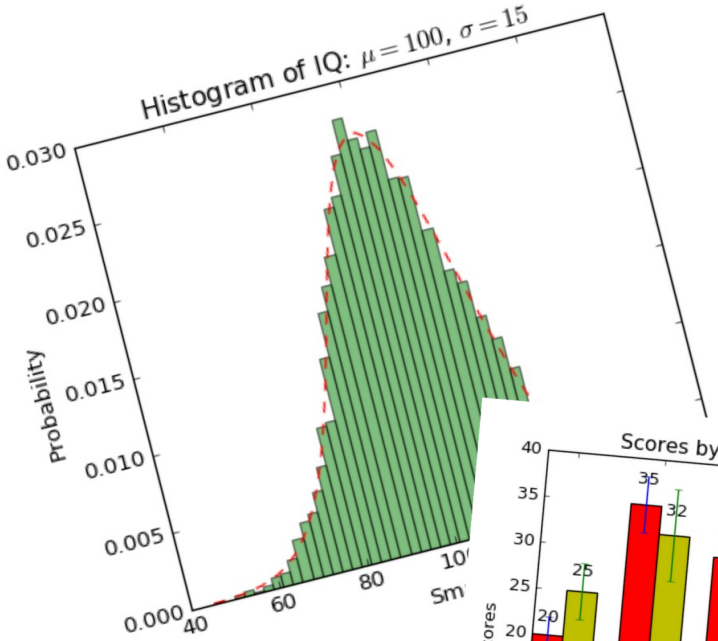
universität wien

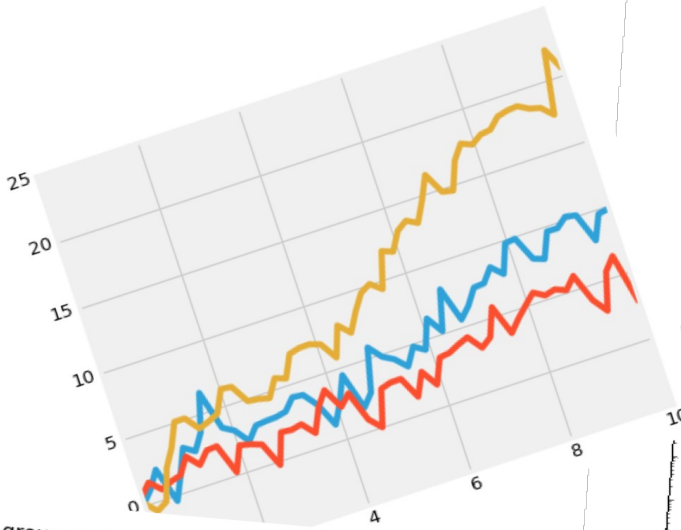
MULTIOMICS DATA SCIENCE – MATPLOTLIB

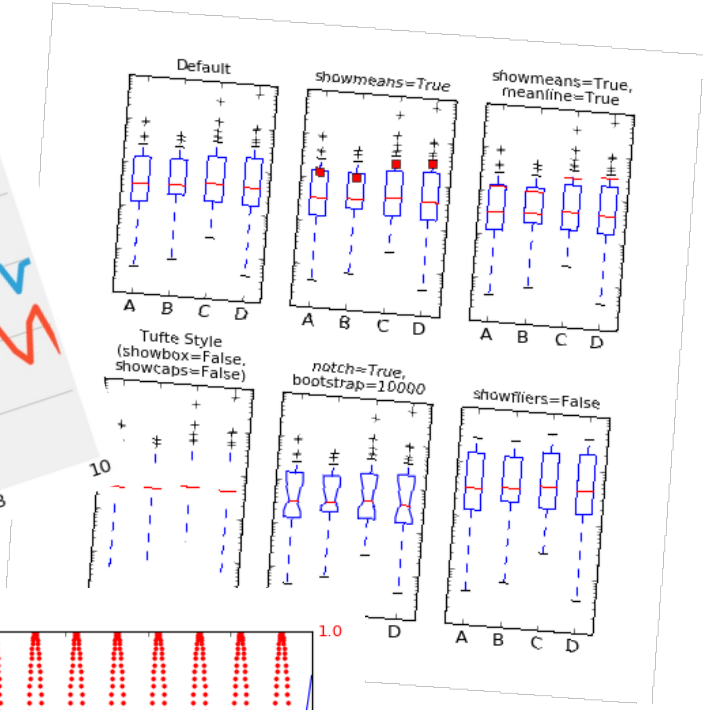
FIGURES / PLOTS / GRAPHS

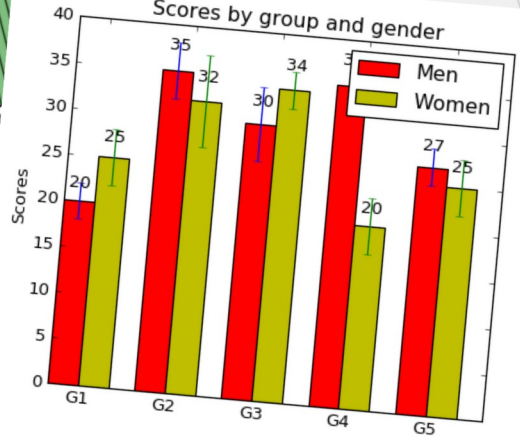
- Figures / plots / graphs offer a form of data visualisation
- For different data, there are different plots for visualisation
- Some plot typse are suited for a good comprehension, some less, some not at all

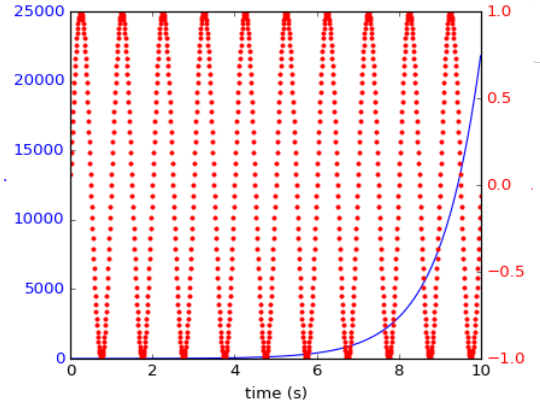
MATPLOTLIB











universität wien

MATPLOTLIB

- Python library for visualisation of big datenstes with plots
- Released in 2003
- Can show figures directly, store as pdf / svg / png, oder embed in a graphical user interface
- Uses NumPy in the background
- Has very big community and is maintained and updated constantly

FIRST PLOT WITH MATPLOTLIB

- Known mechanis for importing libraries:

```
import matplotlib.pyplot as plt
import numpy as np

X = np.random.normal(5, 3, 100) # Please enter here later your actual
Y = np.random.normal(7, 5, 100) # x and y values, no random values

plt.scatter(X, Y)
plt.show()
```

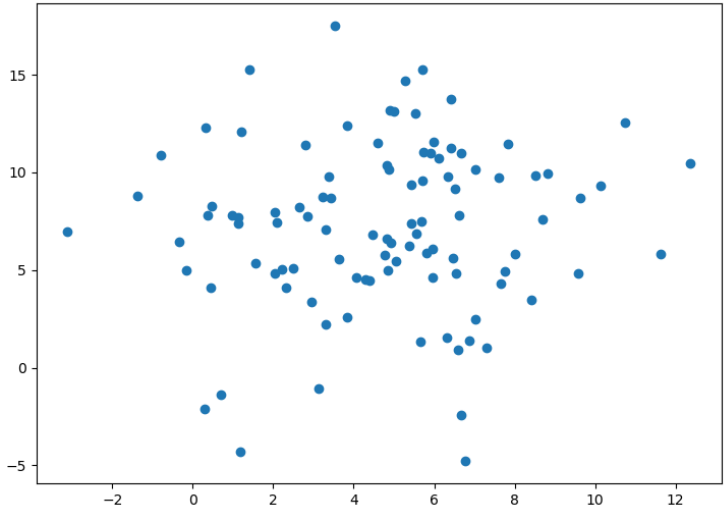
FIRST PLOT WITH MATPLOTLIB

- Known mechanis for importing libraries:

```
import matplotlib.pyplot as plt
import numpy as np

X = np.random.normal(5, 3, 100)
Y = np.random.normal(7, 5, 100)

plt.scatter(X, Y)
plt.show()
```



- The submodule `pyplot` is responsable for the visualization

universität wien

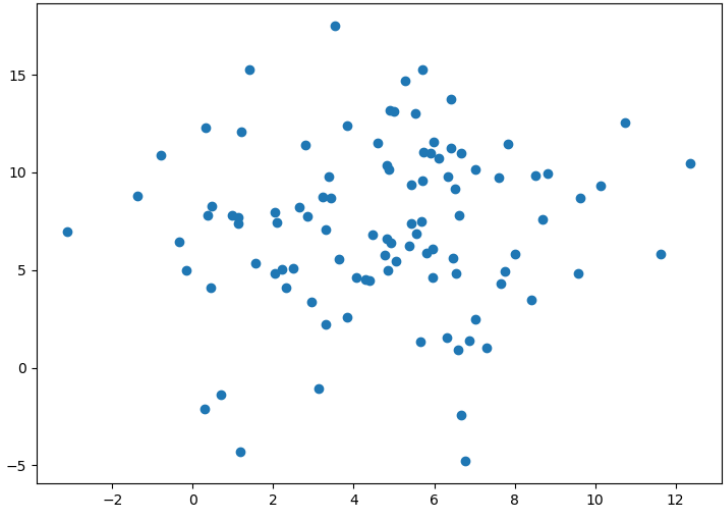
FIRST PLOT WITH MATPLOTLIB

- Known mechanis for importing libraries:

```
import matplotlib.pyplot as plt
import numpy as np

X = np.random.normal(5, 3, 100)
Y = np.random.normal(7, 5, 100)

plt.scatter(X, Y)
plt.show()
```



- The `scatter` function creates a point diagram / scatter plot

universität wien

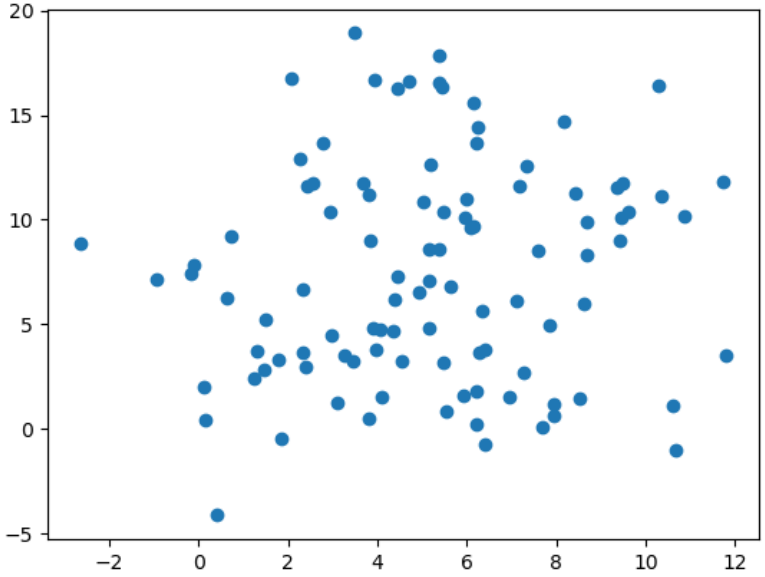
MANIPULATING PLOTS

- When no user-defined parameters are set for the plots, Matplotlib tries to determine good default values
- Such as axis regions, margins between the plot and the window border, font size, etc.
- All these parameter can be defined by the user, too

ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

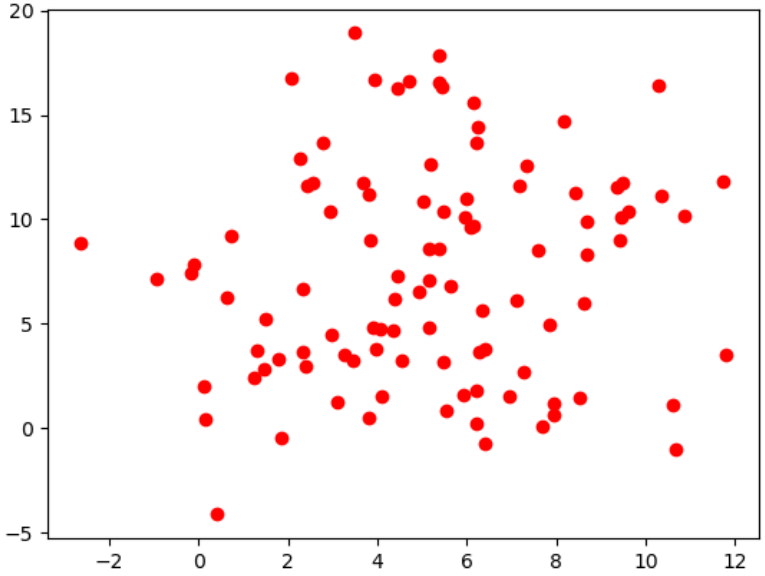
```
plt.scatter(X, Y)
plt.show()
```



ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

```
plt.scatter(X, Y, color = "red")
plt.show()
```

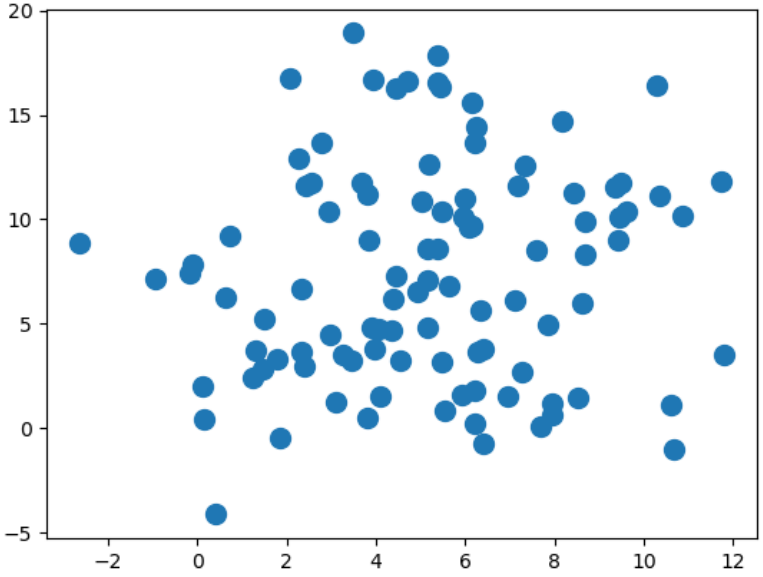


- Colors of the points

ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

```
plt.scatter(X, Y, s = 100)
plt.show()
```



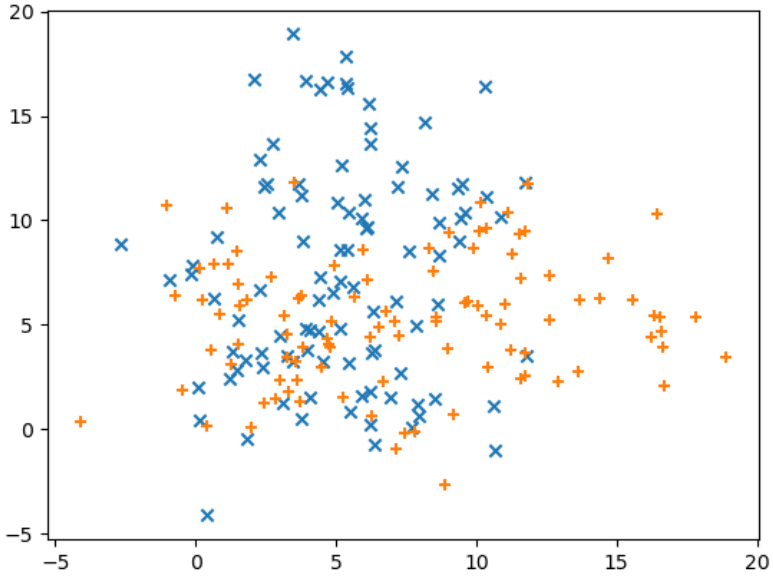
- Point size

universität wien

ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

```
plt.scatter(X, Y, m = "x")
plt.scatter(Y, X, m = "+")
plt.show()
```



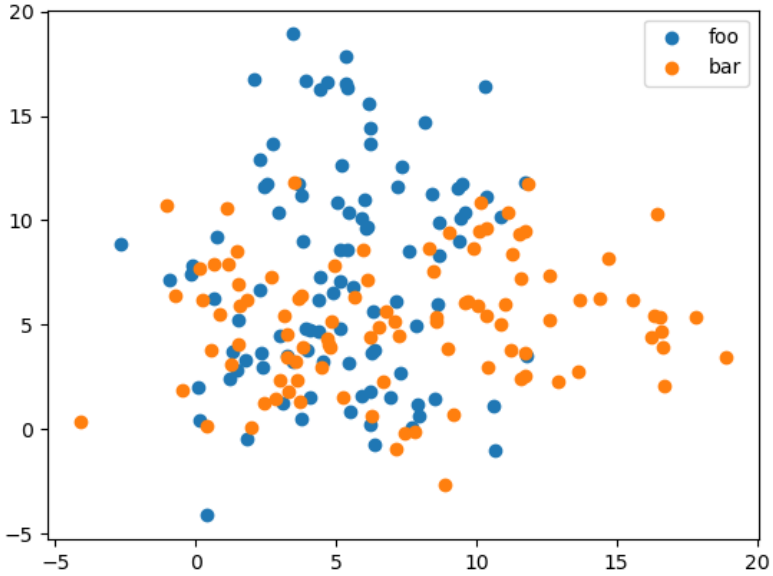
- Point marker symbol

universität wien

ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

```
plt.scatter(X, Y, label = "foo")
plt.scatter(Y, X, label = "bar")
plt.legend()
plt.show()
```



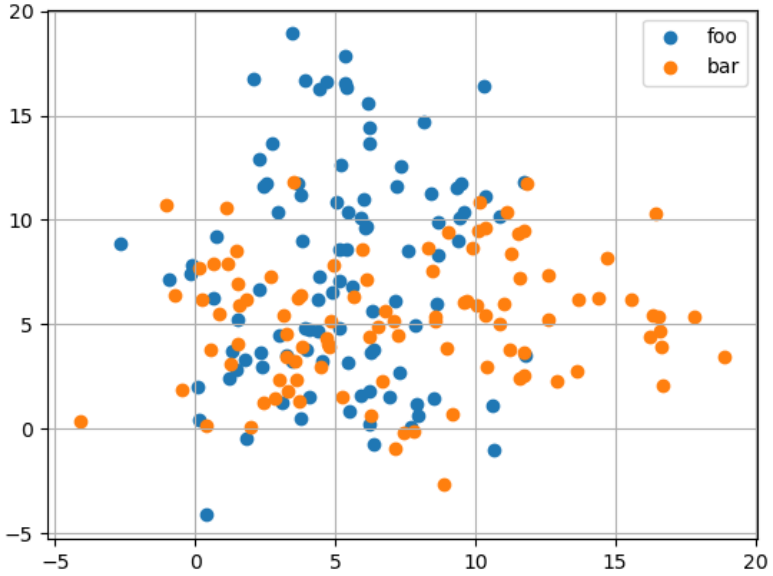
- Legend and label of the points

universität wien

ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

```
plt.scatter(X, Y, label = "foo")
plt.scatter(Y, X, label = "bar")
plt.legend()
plt.grid()
plt.show()
```

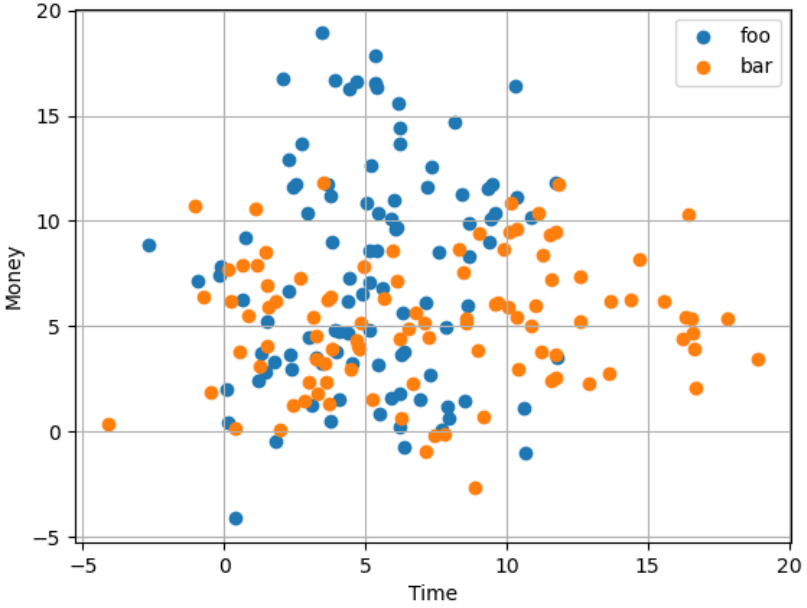


- With a grid in the background

ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

```
plt.scatter(X, Y, label = "foo")
plt.scatter(Y, X, label = "bar")
plt.legend()
plt.grid()
plt.xlabel("Time")
plt.ylabel("Money")
plt.show()
```



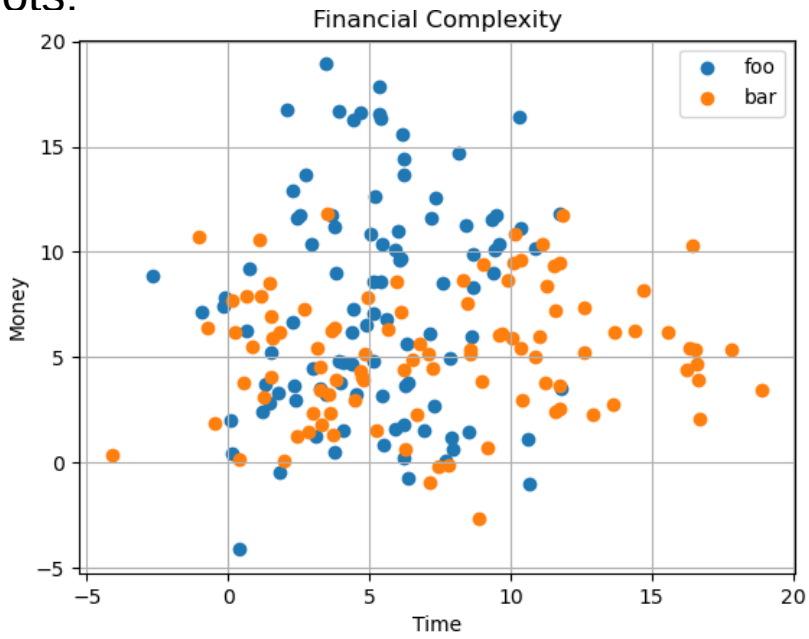
- Axis labels

universität wien

ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

```
plt.scatter(X, Y, label = "foo")
plt.scatter(Y, X, label = "bar")
plt.legend()
plt.grid()
plt.xlabel("Time")
plt.ylabel("Money")
plt.title("Financial Complexity")
plt.show()
```



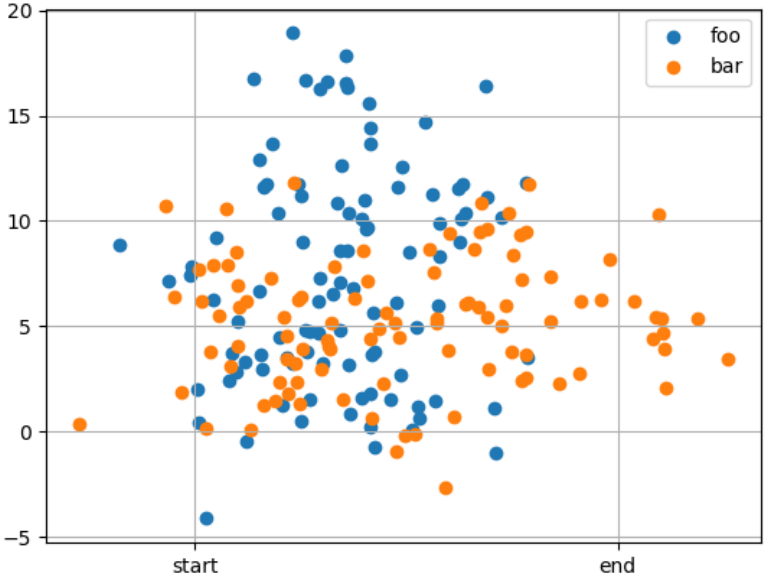
- Plot title

universität wien

ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

```
plt.scatter(X, Y, label = "foo")
plt.scatter(Y, X, label = "bar")
plt.legend()
plt.grid()
plt.xticks([0, 15], ["start", "end"])
plt.show()
```



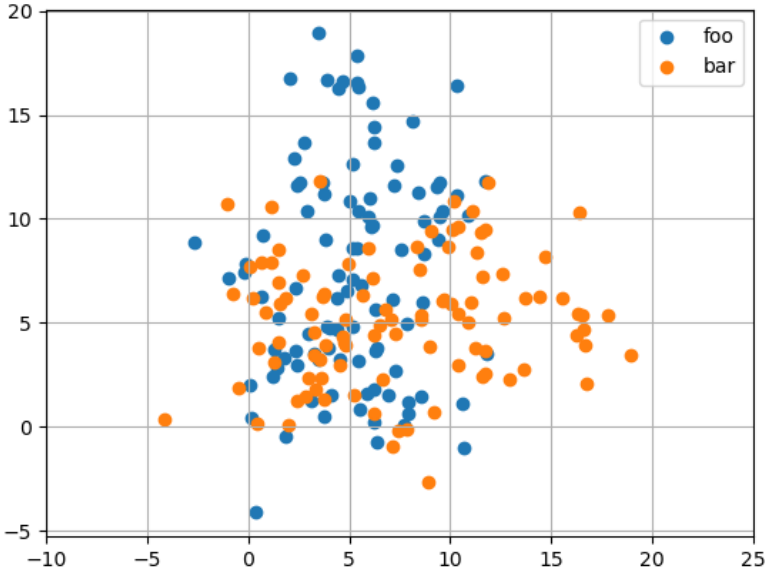
- Adjustment of the axis ticks

universität wien

ADJUSTING PLOTS

- Many functions and parameter to adjust the plots:

```
plt.scatter(X, Y, label = "foo")
plt.scatter(Y, X, label = "bar")
plt.legend()
plt.grid()
plt.xlim(-10, 25)
plt.show()
```

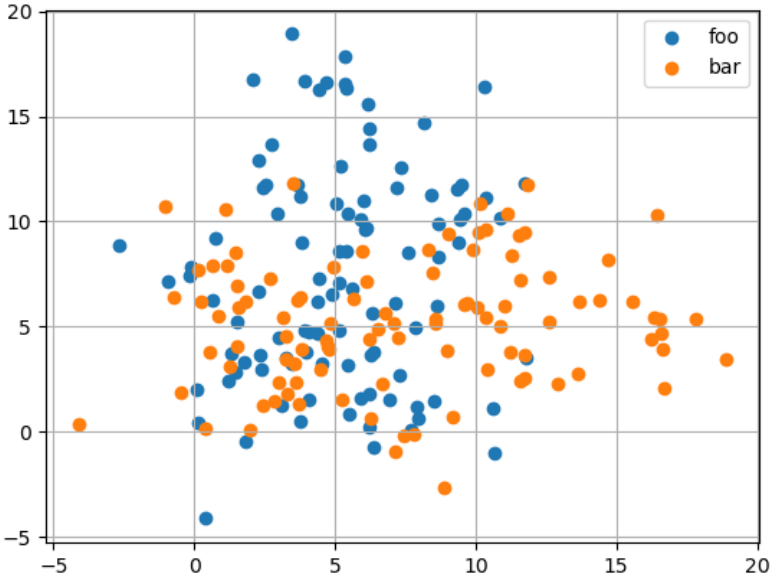


- Adjustment of the axis ranges

SAVING PLOTS

- Plots are stored dependent on the file ending:

```
plt.scatter(X, Y, label = "foo")
plt.scatter(Y, X, label = "bar")
plt.legend()
plt.grid()
plt.xticks([0, 15], ["start", "end"])
plt.savefig("myplot.pdf")
plt.savefig("myplot.png", dpi = 300)
```



- The parameter dots per inch `dpi` is optional

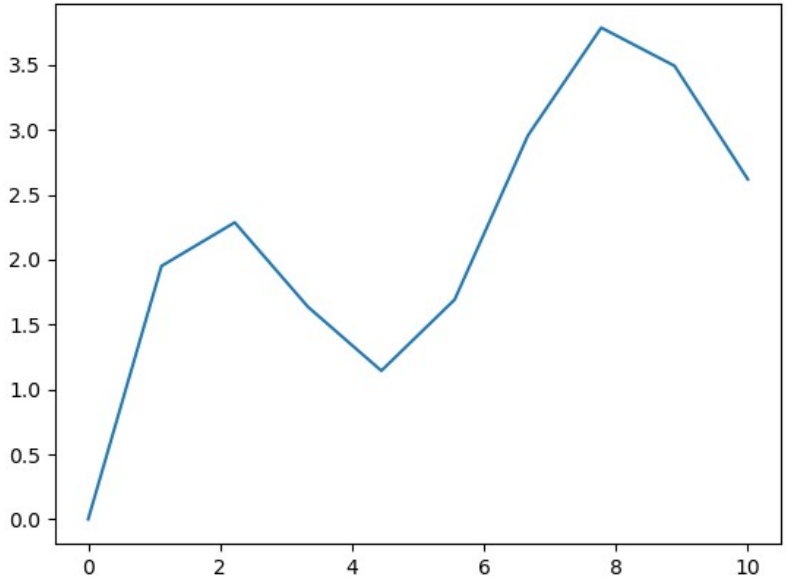
universität wien

FURTHER PLOT TYPES

- Plot functions by using the line `plot` function, create equally distributed x values

```
X = np.arange(0, 10, 1)
Y = np.sin(X) + np.sqrt(X)

plt.plot(X, Y)
plt.show()
```

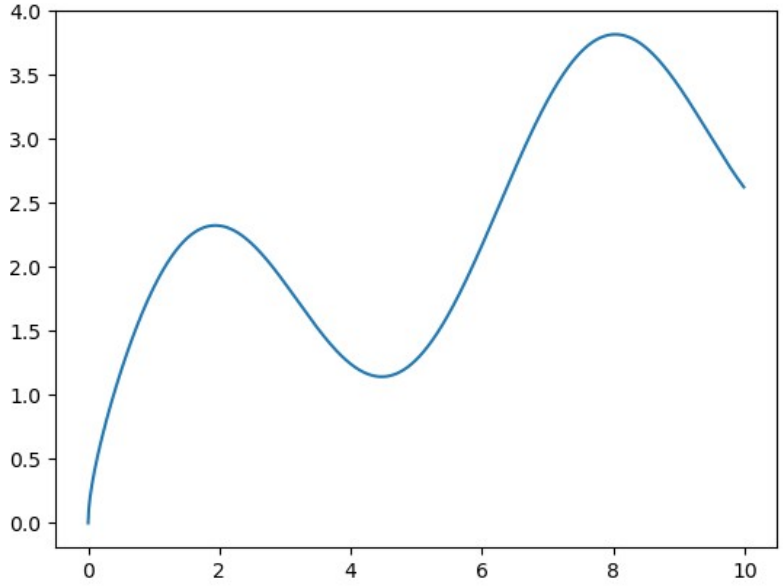


FURTHER PLOT TYPES

- The more x values you create, the smoother the curve will look like

```
X = np.arange(0, 10, 0.1)
Y = np.sin(X) + np.sqrt(X)

plt.plot(X, Y)
plt.show()
```

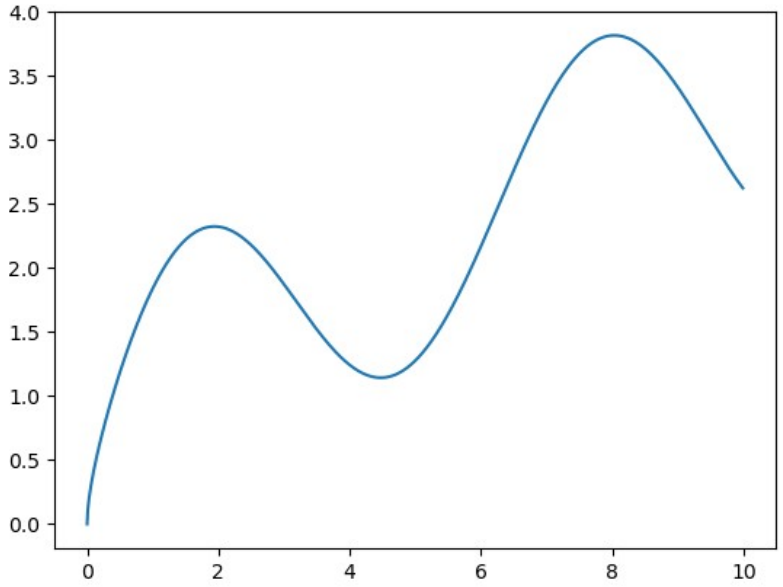


FURTHER PLOT TYPES

- Plot functions by using the line `plot` function, define your mathematical function

```
X = np.arange(0, 10, 0.1)
Y = np.sin(X) + np.sqrt(X)

plt.plot(X, Y)
plt.show()
```

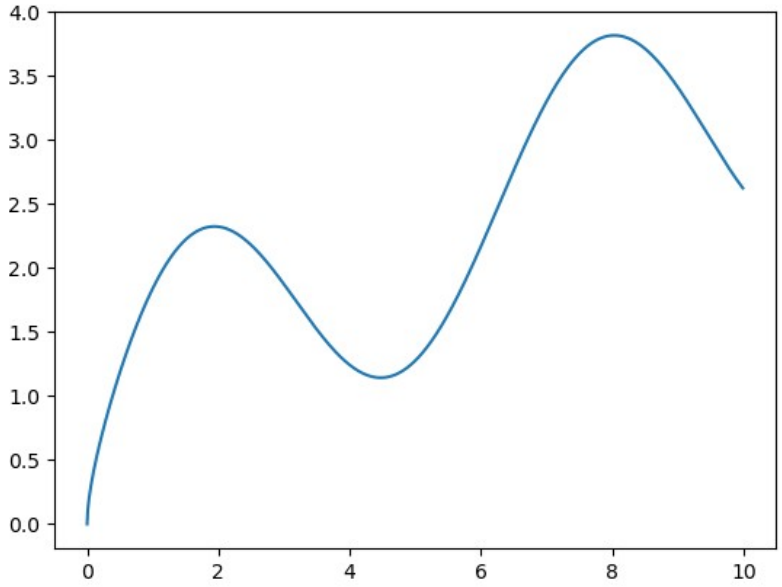


FURTHER PLOT TYPES

- Plot functions by using the line `plot` function

```
X = np.arange(0, 10, 0.1)
Y = np.sin(X) + np.sqrt(X)

plt.plot(X, Y)
plt.show()
```

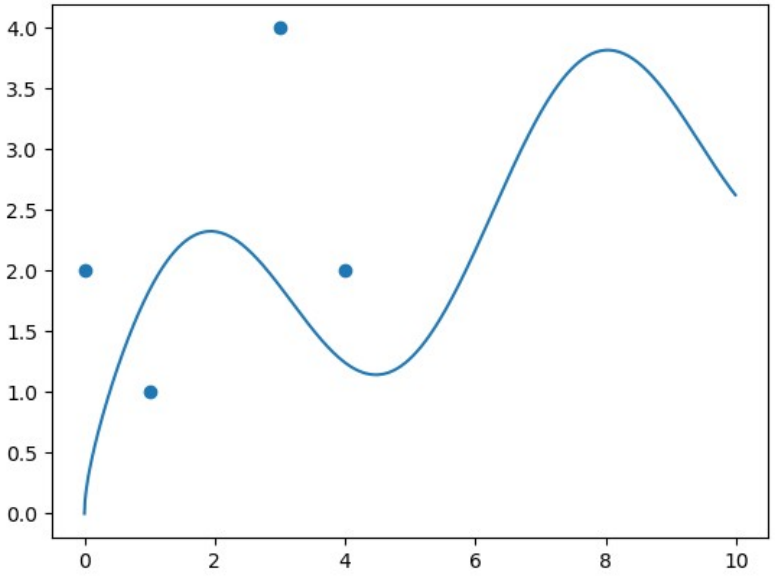


FURTHER PLOT TYPES

- Or different plots combined with each other

```
X = np.arange(0, 10, 0.1)
Y = np.sin(X) + np.sqrt(X)

plt.plot(X, Y)
plt.scatter([0, 3, 4, 1], [2, 4, 2, 1])
plt.show()
```

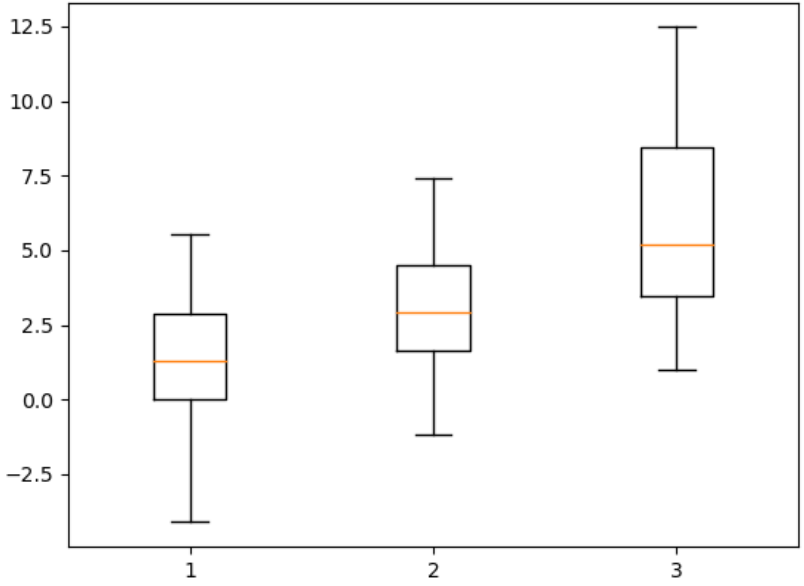


FURTHER PLOT TYPES

- Box plots with the `boxplot` function

```
X = [np.random.normal(1, 2, 50),
     np.random.normal(4, 3, 30),
     np.abs(np.random.normal(3, 5, 20))]

plt.boxplot(X)
plt.show()
```

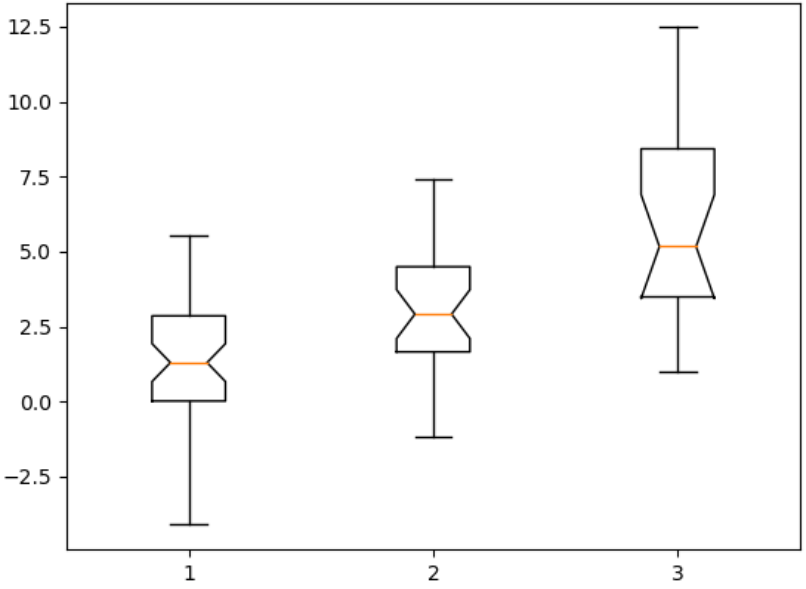


FURTHER PLOT TYPES

- Box plots with the `boxplot` function

```
X = [np.random.normal(1, 2, 50),
     np.random.normal(4, 3, 30),
     np.abs(np.random.normal(3, 5, 20))]

plt.boxplot(X, notch = True)
plt.show()
```



- With the notch parameter

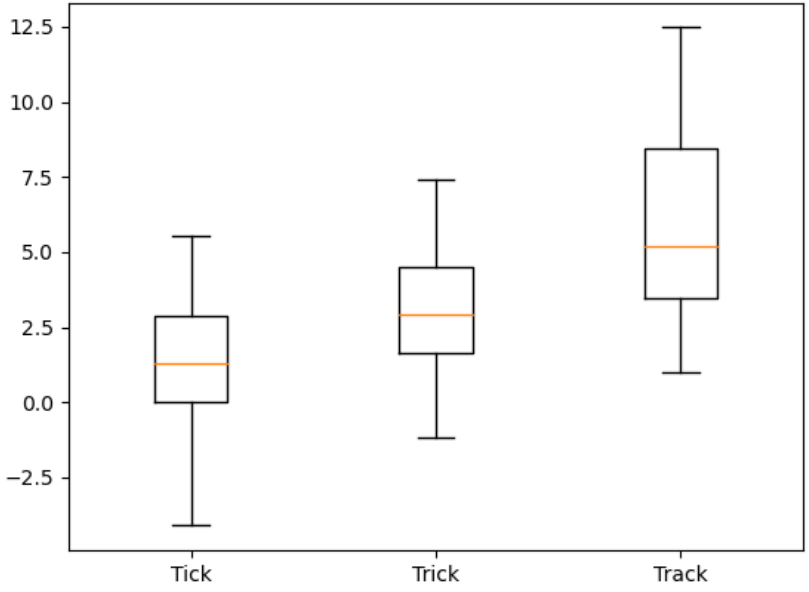
universität wien

FURTHER PLOT TYPES

- Box plots with the `boxplot` function

```
X = [np.random.normal(1, 2, 50),
     np.random.normal(4, 3, 30),
     np.abs(np.random.normal(3, 5, 20))]

l = ["Tick", "Trick", "Track"]
plt.boxplot(X, labels = l)
plt.show()
```



- With labels

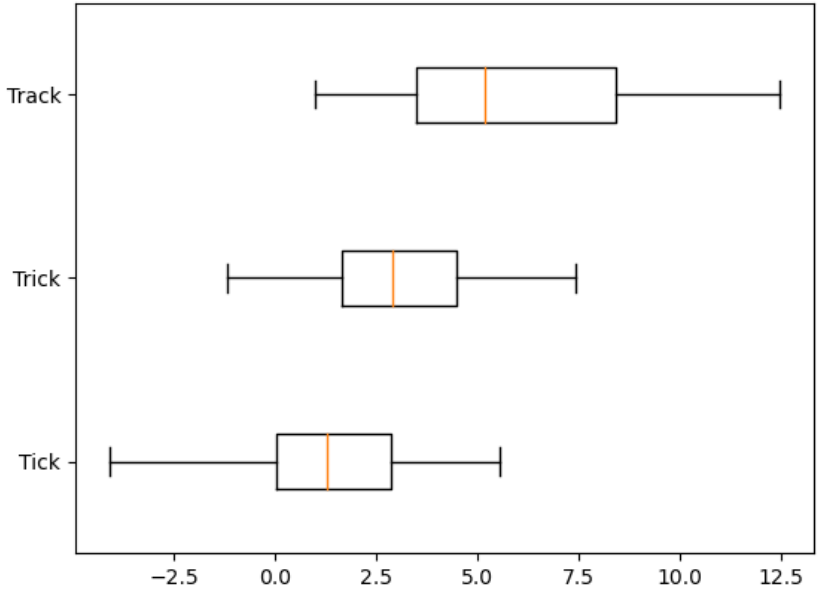
universität wien

FURTHER PLOT TYPES

- Box plots with the `boxplot` function

```
X = [np.random.normal(1, 2, 50),
     np.random.normal(4, 3, 30),
     np.abs(np.random.normal(3, 5, 20))]

l = ["Tick", "Trick", "Track"]
plt.boxplot(X, labels = l, vert = False)
plt.show()
```



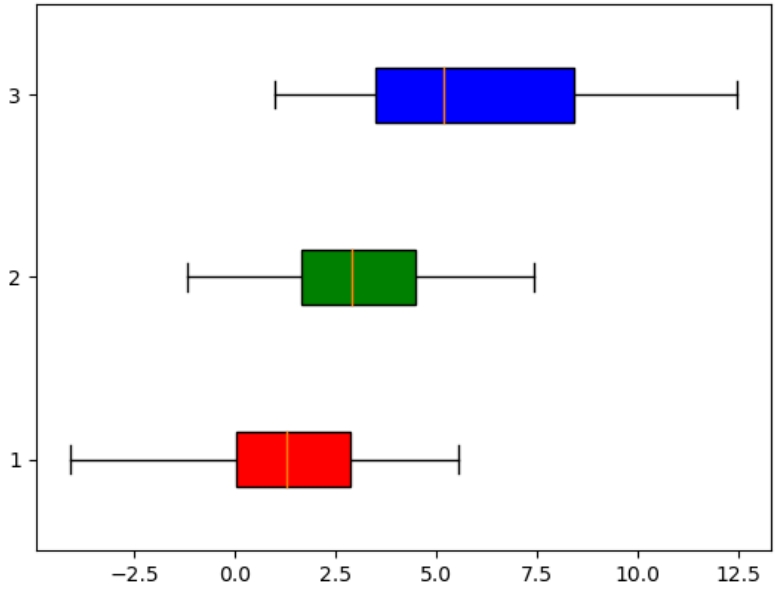
- Horizontal, or not vertical

universität wien

FURTHER PLOT TYPES

- Box plots with the `boxplot` function

```
X = [np.random.normal(1, 2, 50),
     np.random.normal(4, 3, 30),
     np.abs(np.random.normal(3, 5, 20))]
col = ["red", "green", "blue"]
bplot = plt.boxplot(X, vert = False,
                    patch_artist = True)
for box, c in zip(bplot["boxes"], col):
    box.set_facecolor(c)
plt.show()
```



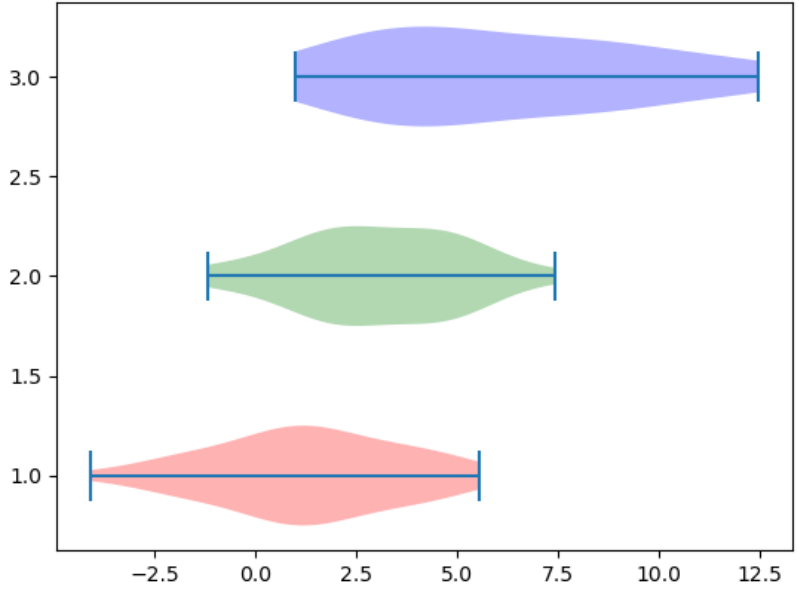
- Colored

universität wien

FURTHER PLOT TYPES

- Violin plots with the `violinplot` function

```
X = [np.random.normal(1, 2, 50),
     np.random.normal(4, 3, 30),
     np.abs(np.random.normal(3, 5, 20))]
col = ["red", "green", "blue"]
vplot = plt.violinplot(X, vert = False)
for box, c in zip(vplot["bodies"], col):
    box.set_facecolor(c)
plt.show()
```

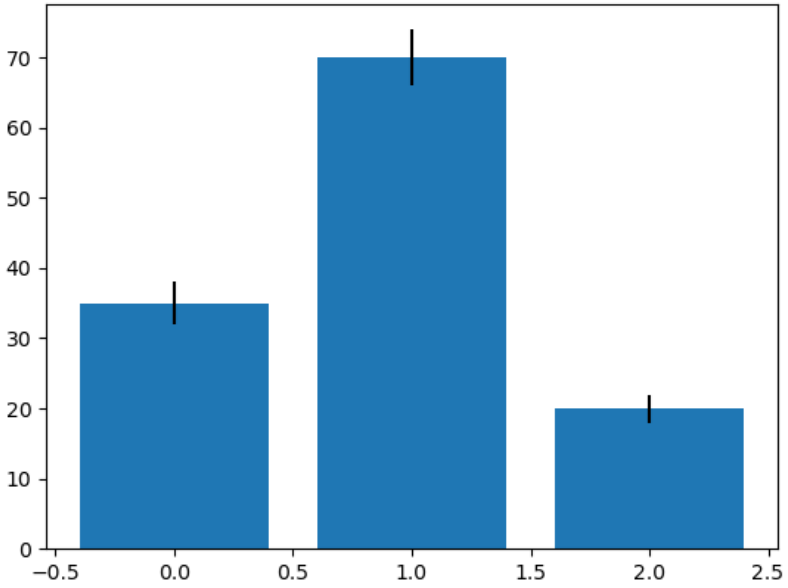


FURTHER PLOT TYPES

- Bar plots with the `barplot` function

```
prot = [‘PPARG‘, ‘SHH‘, ‘WHAMM‘]
u_m, u_s = [35, 70, 20], [3, 4, 2]
t_m, t_s = [20, 40, 15], [4, 3, 1]

x = np.arange(len(prot))
plt.bar(x, u_m, yerr = u_s)
plt.show()
```



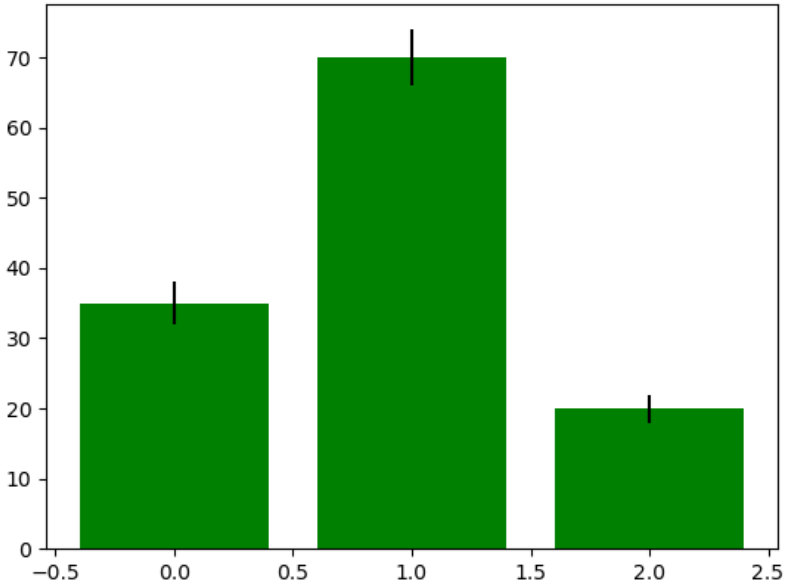
universität wien

FURTHER PLOT TYPES

- Bar plots with the `barplot` function

```
prot = [‘PPARG‘, ‘SHH‘, ‘WHAMM‘]
u_m, u_s = [35, 70, 20], [3, 4, 2]
t_m, t_s = [20, 40, 15], [4, 3, 1]

x = np.arange(len(prot))
plt.bar(x, u_m, yerr = u_s, color = ‘g‘)
plt.show()
```



- Colored

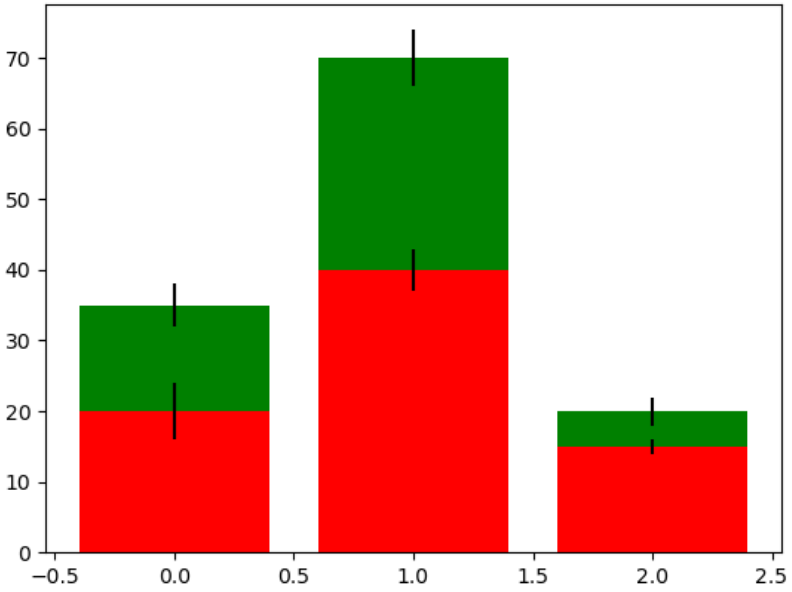
universität wien

FURTHER PLOT TYPES

- Bar plots with the `barplot` function

```
prot = [‘PPARG‘, ‘SHH‘, ‘WHAMM‘]
u_m, u_s = [35, 70, 20], [3, 4, 2]
t_m, t_s = [20, 40, 15], [4, 3, 1]

x = np.arange(len(prot))
plt.bar(x, u_m, yerr = u_s, color = ‘g‘)
plt.bar(x, t_m, yerr = t_s, color = ‘r‘)
plt.show()
```



- Several bar plots

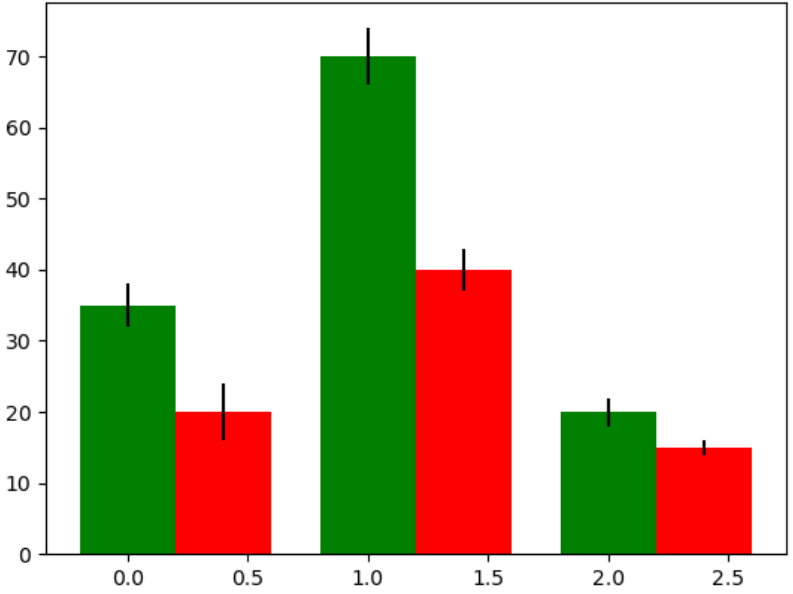
universität wien

FURTHER PLOT TYPES

- Bar plots with the `barplot` function

```
prot = [‘PPARG‘, ‘SHH‘, ‘WHAMM‘]
u_m, u_s = [35, 70, 20], [3, 4, 2]
t_m, t_s = [20, 40, 15], [4, 3, 1]

x, w = np.arange(len(prot)), 0.4
plt.bar(x, u_m, w, yerr = u_s, color = 'g')
plt.bar(x + w, t_m, w, yerr = t_s, color = 'r')
plt.show()
```



- Adjust width

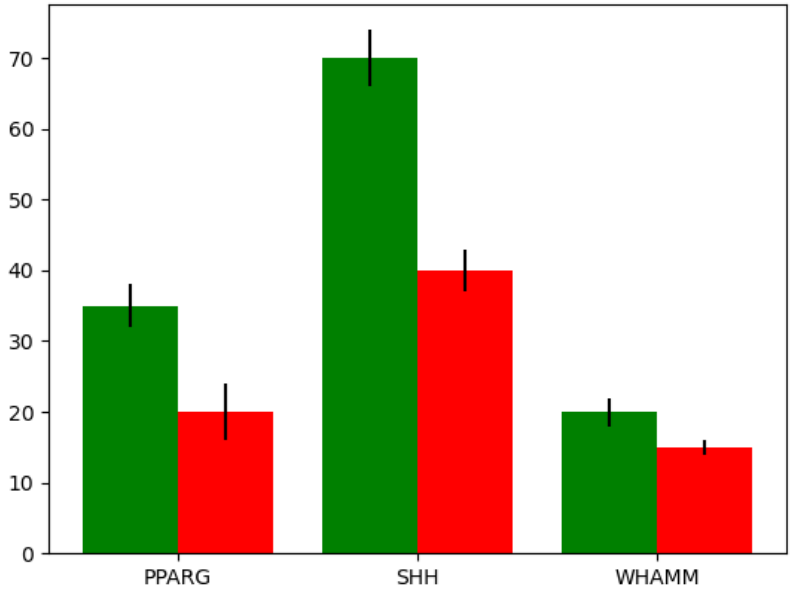
universität wien

FURTHER PLOT TYPES

- Bar plots with the `barplot` function

```
prot = [‘PPARG‘, ‘SHH‘, ‘WHAMM‘]
u_m, u_s = [35, 70, 20], [3, 4, 2]
t_m, t_s = [20, 40, 15], [4, 3, 1]

x, w = np.arange(len(prot)), 0.4
plt.bar(x, u_m, w, yerr = u_s, color = 'g')
plt.bar(x + w, t_m, w, yerr = t_s, color = 'r')
plt.xticks(x + w / 2, prot)
plt.show()
```



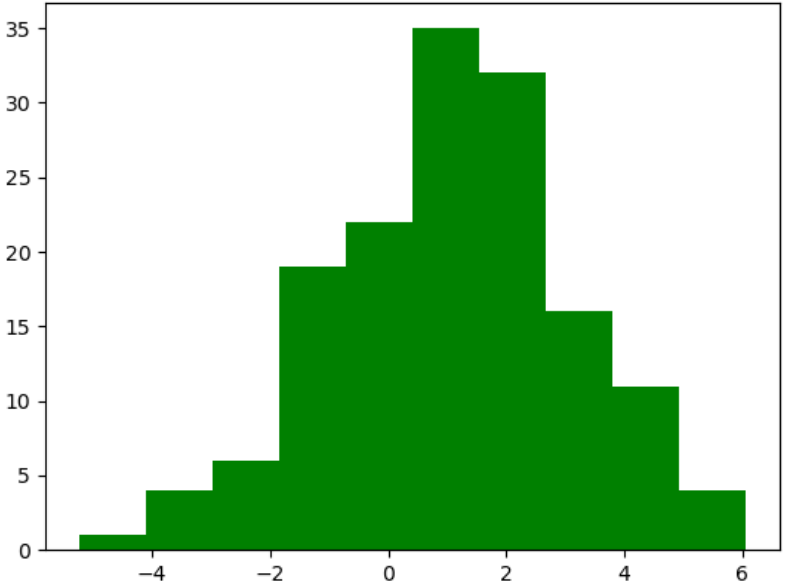
- With correct labels

universität wien

FURTHER PLOT TYPES

- Histograms with the `hist` function

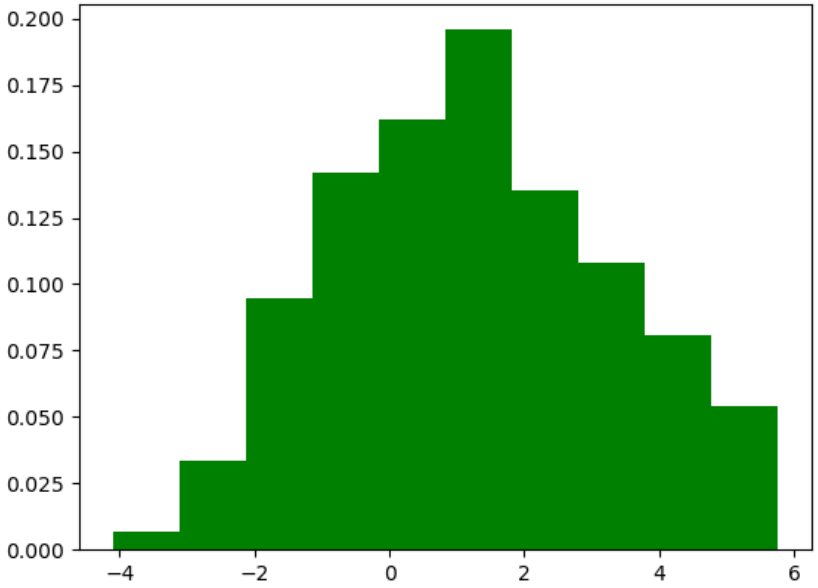
```
X = np.random.normal(1, 2, 150)
plt.hist(X, color = "g")
plt.show()
```



FURTHER PLOT TYPES

- Histograms with the `hist` function, with normalization (area under curve equals 1)

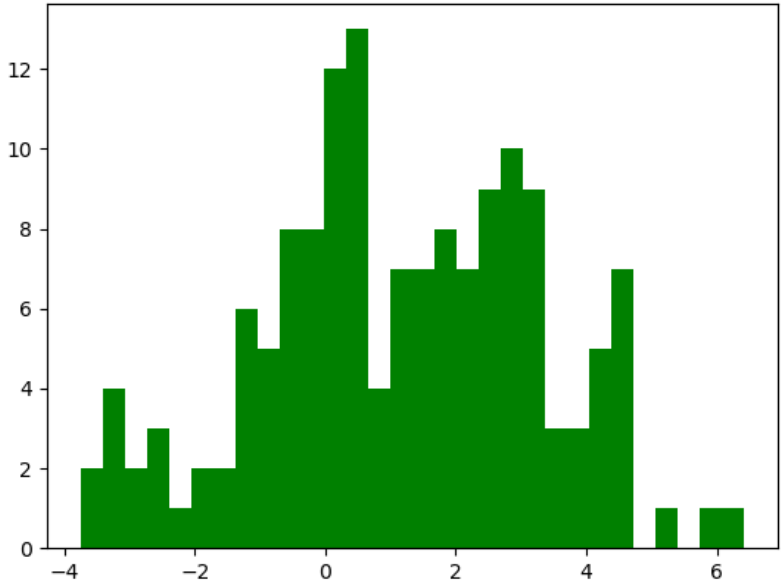
```
X = np.random.normal(1, 2, 150)
plt.hist(X, density = 1, color = "g")
plt.show()
```



FURTHER PLOT TYPES

- Histograms with the `hist` function

```
X = np.random.normal(1, 2, 150)
plt.hist(X, bins = 30, color = "g")
plt.show()
```

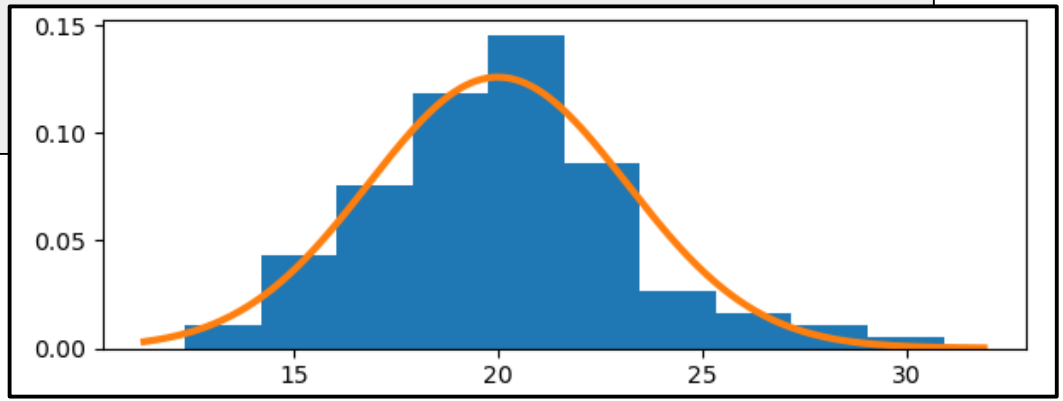


FURTHER PLOT TYPES

- Histograms with the `hist` function

```
def normal_dist(x, m, s):
    return np.exp(-0.5 * ((x - m) / s)**2) / (s * np.sqrt(2 * np.pi))

data = np.random.normal(20, 3, 100)   # Use your TRUE data here
mue, sigma = data.mean(), data.std()
x = np.arange(data.min() - 2 * sigma, data.max() + 2 * sigma, 0.01)
y = normal_dist(x, mue, sigma)
plt.hist(data, density = True)
plt.plot(x, y, lw = 3)
plt.show()
```



- Normalization important

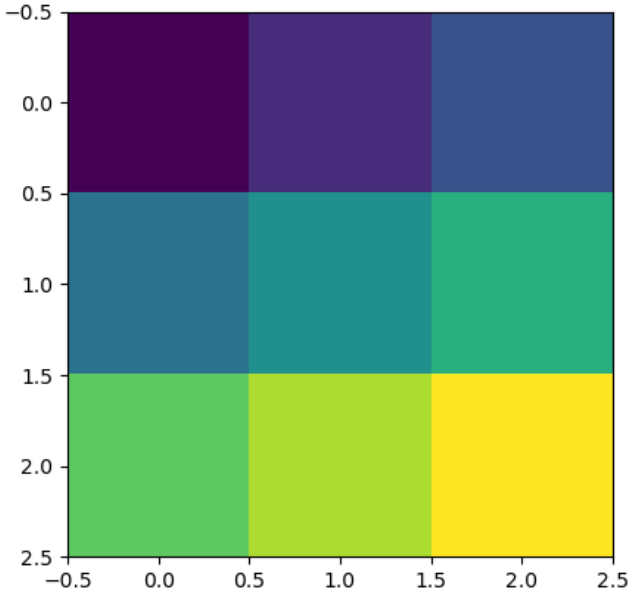
universität wien

FURTHER PLOT TYPES

- Heatmaps with the `imshow` function

```
X = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

plt.imshow(X)
plt.show()
```



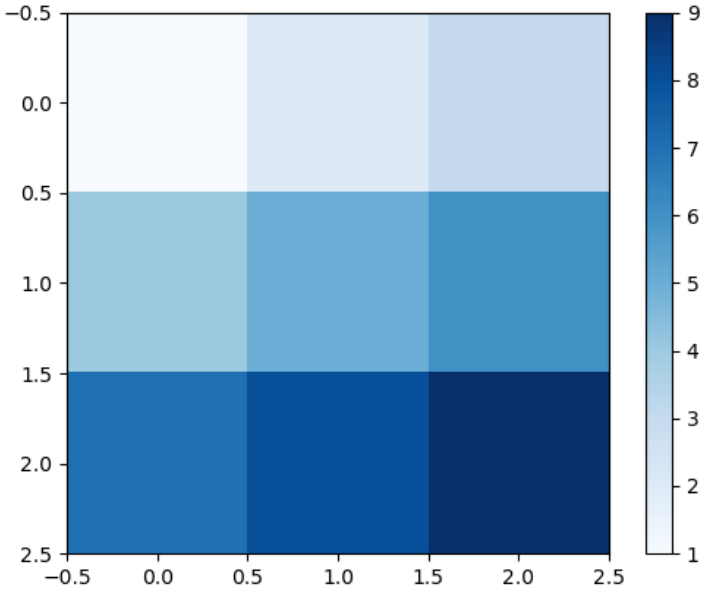
universität wien

FURTHER PLOT TYPES

- Heatmaps with the `imshow` function

```
X = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

hm = plt.imshow(X, cmap = "Blues")
plt.colorbar(hm)
plt.show()
```



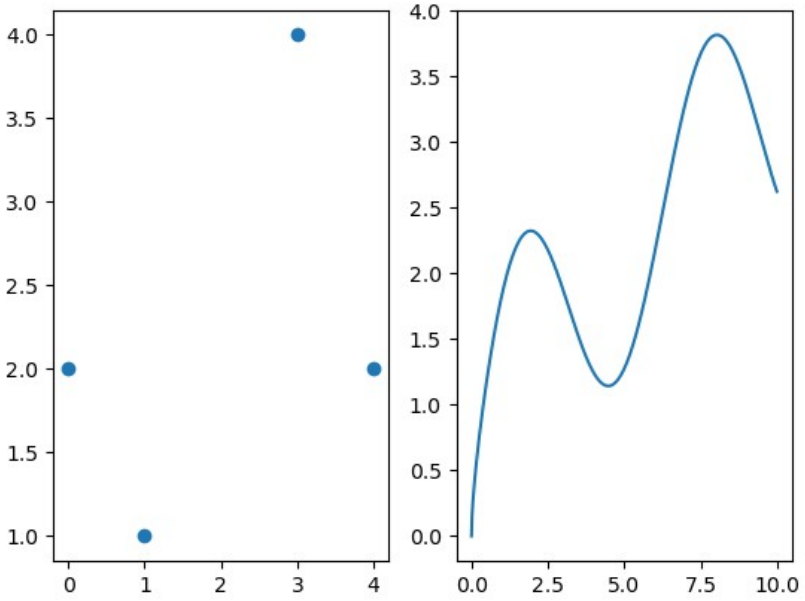
- Color scale of the heatmap kann be adjusted and shown with the `colorbar` function

universität wien

SUBPLOTS

- Show several subplots in one plot:

```
fig, ax = plt.subplots(1, 2)
X = np.arange(0, 10, 0.01)
Y = np.sin(X) + np.sqrt(X)
ax[0].scatter([0, 3, 4, 1], [2, 4, 2, 1])
ax[1].plot(X, Y)
plt.show()
```



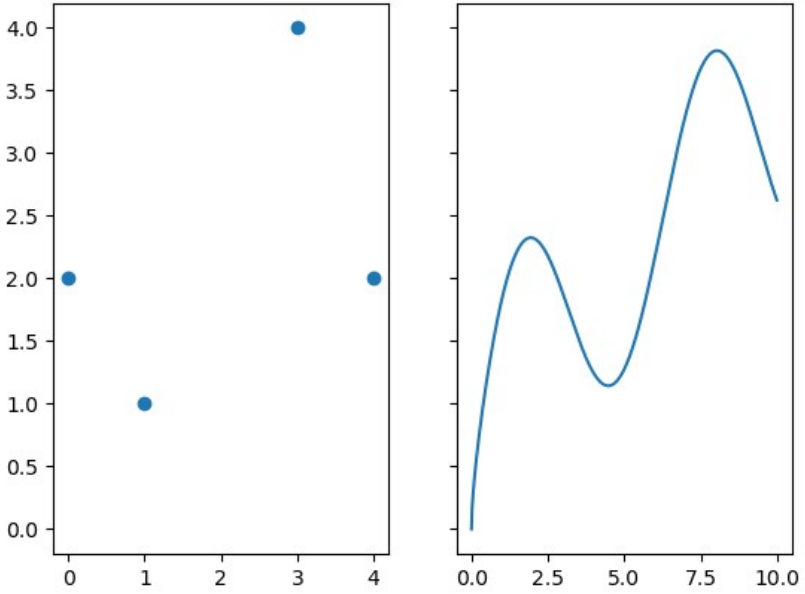
- Instead of assigning all data plots to the object `plt`, assignment happens on the axes `ax[0]` and `ax[1]`

universität wien

SUBPLOTS

- Show several subplots in one plot:

```
fig, ax = plt.subplots(1, 2,
          sharey = True)
X = np.arange(0, 10, 0.01)
Y = np.sin(X) + np.sqrt(X)
ax[0].scatter([0, 3, 4, 1], [2, 4, 2, 1])
ax[1].plot(X, Y)
plt.show()
```



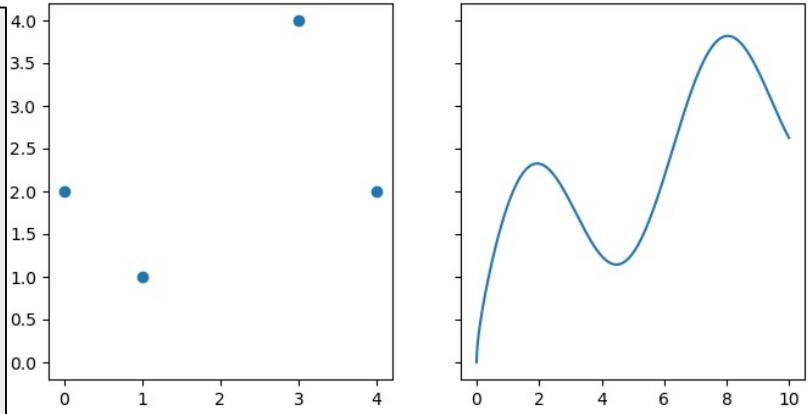
- Share y axis

universität wien

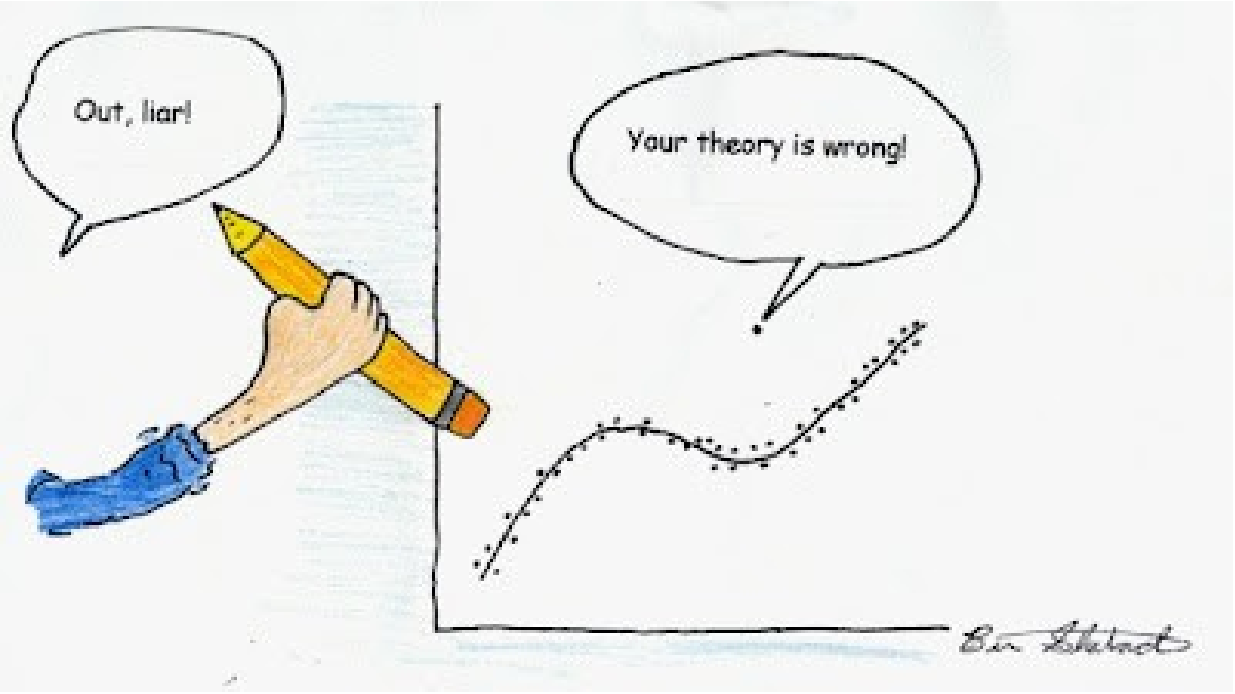
SUBPLOTS

- Show several subplots in one plot:

```
fig, ax = plt.subplots(1, 2,
          sharey = True,
          figsize = (8, 4))
X = np.arange(0, 10, 0.01)
Y = np.sin(X) + np.sqrt(X)
ax[0].scatter([0, 3, 4, 1], [2, 4, 2, 1])
ax[1].plot(X, Y)
plt.show()
```



- Adjust plot size

DOCUMENTATION

- Comprehensive documentations, tutorials, and examples you will find here:
  - https://matplotlib.org
  - https://google.de
  - https://youtube.com